Notes on the TORTOISE System

by
Ivan Gaydardzhiev

ABSTRACT

TORTOISE is a complete computer implemented in software. It defines a 32-bit processor, instruction set, virtual memory, interrupts, memory-mapped devices, boot ROM, assembler, linker, C compiler, and Unix-like operating system. The processor preserves its original 16-bit-oriented instruction set while extended operation adds paging, privilege separation, and 32-bit instructions. The toolchain produces relocatable TOF objects and TEX1 executables. The operating system implements processes, copy-on-write fork, demand paging, signals, pipes, a virtual file system, and a command shell. The compiler builds itself, the assembler, and the linker for execution inside the emulated system; its in-system assembly output is byte-identical to output generated on the host. These notes describe the machine architecture, instruction execution, toolchain, kernel, storage, runtime, and self-hosting tests.

1. Introduction

TORTOISE is a computer whose processor, physical memory, controllers, and storage interfaces are C programs. The host runs the `tortoise` executable, which loads a program or starts the boot ROM. The guest executes its own instruction set rather than the host's machine instructions. It can run a flat assembly program, a linked C executable, or a disk image containing a kernel and a file system. The same machine instructions support these three uses.

The processor has eight general-purpose 32-bit registers, a program counter, a stack pointer, condition flags, and privileged control registers. Its original instructions operate principally on 16-bit values; the extended instruction set supplies 32-bit arithmetic, relative branches, byte and word accesses, an exception mechanism, address translation, and an interrupt controller. Both generations of instructions execute in the same emulator. The original programs retain their binary encodings and expected output.

TORTOISE includes an assembler named `as`, a linker named `tlink`, and a C compiler named `tcc0`. C source is preprocessed, parsed into typed expressions and statements, written as TORTOISE assembly, assembled into a relocatable TOF object, and linked into a TEX1 executable. A small C library supplies startup code, memory allocation, formatted I/O, and operating-system calls. A separate bare-machine version of the library uses the machine devices directly.

The operating system boots from the emulated disk through the ROM and a 512-byte boot sector. Its kernel establishes paging, handles faults and interrupts, schedules processes, accesses disk files through a virtual file system, and starts `/bin/init` and a shell. It compiles and runs C source from within the guest. The assembler, linker, and compiler are themselves C programs compiled by `tcc0`, which closes the toolchain bootstrap inside TORTOISE.

2. Structure

The host emulator is divided among `src/cpu.c`, `src/dev.c`, `src/dis.c`, `src/emu.c`, `src/opts.c`, `src/dbg.c`, `src/gdb.c`, and the corresponding headers in `include`. CPU execution and address translation belong to `cpu.c`. Device registers, UART streams, disk transfers, timers, and the interrupt controller belong to `dev.c`. The emulator driver handles loading and run options; the disassembler, interactive debugger, and GDB remote stub inspect the same CPU state.

`assembler/assembler.c` reads TORTOISE assembly and produces flat binaries or TOF objects. `tools/tlink.c` resolves symbols and relocations to produce a TEX1 executable or a flat image. The compiler is divided into `cc/pp.c`, `cc/parse.c`, `cc/gen.c`, and `cc/main.c`: preprocessing, parsing and types, assembly generation, and the driver. `lib/` contains the runtime sources, headers, and startup assembly. The ROM is written in `rom/rom.asm`.

The guest system is under `os/`. `os/boot` contains the disk boot sector, `os/kernel` contains the kernel, and `os/bin` contains its user programs. The system file creator is `tools/tdisk.c`. The top-level `Makefile` connects these components: it builds the tools on the host, uses the TORTOISE compiler for the kernel and its user programs, constructs a disk image, and runs that image through the emulator.

```
make
make os
make run_os
make selfhost.img
make run_selfhost_test
```

Each command uses the tools built by the same project. The first produces the emulator and toolchain. The operating-system targets add the ROM, kernel, disk, and programs. The self-hosting target places the compiler, assembler, linker, headers, and libraries on a guest disk so that the system can rebuild software from inside its own shell.

2.1. Buffers and machine state

`CPU` holds `PC`, the eight-element `registers` array, `SP`, flags, mode, interrupt state, control registers, RAM pointer and size, current instruction address, and a 64-bit instruction count. RAM is allocated on the host and is addressed with guest physical addresses. `cpu_init_size` zeros the machine state, allocates the requested RAM, initializes `SP` to its upper end, places the RAM size in `CR8`, and resets the devices. The normal RAM range is 1 to 256 MB.

The assembler keeps dynamically grown arrays of symbols, lines, macros, relocations, and section bytes. The linker keeps object records, a hash table of global symbols, and the layout of text, data, and BSS. The compiler allocates tokens, types, symbols, expression trees, functions, initializers, and string literals. These representations are intentionally separate: CPU state describes a running machine; a TOF record describes machine code before final addresses have been assigned.

Device state includes interrupt pending and mask bits, timer counters, UART buffers, disk commands, and optional SDL input and framebuffer state. Saving the emulator records the registers, control state, RAM, device registers, and executed instruction count. Loading such a state restores execution at that machine point. The backing disk remains a separate persistent image.

3. The processor

The instruction set is defined by the opcode declarations in `include/cpu.h`, the instruction descriptions in `src/dis.c`, and execution in `src/cpu.c`. A register byte normally packs its two operands into four bits each. All multi-byte values in instructions, RAM, TOF records, and TEX1 headers are little-endian. The guest machine is 32-bit even when the host program runs on a 64-bit processor.

3.1. Registers and flags

`R0` through `R7` are 32-bit general-purpose registers. `PC` names the next instruction and `SP` names the stack top; the stack grows toward lower addresses. `FLAGS` contains `Z` for zero, `C` for carry or borrow, `S` for sign, `V` for signed overflow, `I` for enabled interrupts, and `U` for user mode. The emulator derives the flag word from the stored status fields through `cpu_get_flags` and restores it through `cpu_set_flags`.

The privileged control registers have distinct functions. `CR0` selects extended operation, base-and-limit protection, and paging. `CR1` gives the exception vector table. `CR2` holds the last faulting address. `CR3` holds the page-directory base; `CR4` is the supervisor stack used on entry from user mode. `CR5` and `CR6` hold the protected-mode base and limit. `CR7` records the exception vector and error code; `CR8` reports installed RAM. `MFCR` and `MTCR` move values between those registers and ordinary registers.

3.2. Legacy instructions

The original instruction set remains present at opcodes below `0x41`, with `HALT` at `0xFF`. It includes 16-bit immediate loads and arithmetic, direct loads and stores, conditional and unconditional jumps, subroutine calls, an interrupt instruction, port I/O, and the original stack operations. The original `CALL` and `RET` store and restore 16-bit return addresses. The 32-bit immediate arithmetic and load/store instructions at opcodes `0x20` through `0x2B` supplement these operations without changing their older encodings.

```
LOAD R0, 5
loop:
SUB R0, 1
JNZ loop
HALT
```

This is a complete legacy program. The assembler emits a flat binary, and the emulator begins it at address zero. The regression corpus includes logic gates, bit masking, arithmetic, flags, subroutines, indirect accesses, and interrupt tests from the original instruction set. The legacy compatibility test compares both the binary files and their output with the recorded baselines.

3.3. Instruction formats

The extended encoding has five formats. `N` consists of a one-byte opcode. `R` adds a register byte. `RI` carries a register byte followed by a 32-bit immediate. `I` contains a register-selection byte and a 32-bit target or immediate. `M` contains a register byte and a signed 32-bit displacement. The common lengths are one, two, and six bytes.

For memory operations the base field identifies `R0` through `R7`, or value eight for `SP`. In a relative branch, the displacement is added to the address immediately following the

instruction. This convention applies to `JMPREL`, `CALLREL`, conditional branches, and `LEAPC`. It permits the assembler and linker to place separately compiled code without encoding a full absolute destination in every instruction.

```
LEAPC R0, message
CALLREL print
JMPREL done
message: .asciiz "TORTOISE"
done: HALT
```

`src/dis.c` gives opcodes their names, operand formats, and encoded lengths. The assembler selects the encoding from its instruction table, while the emulator uses the length table when fetching an instruction. Both parts therefore describe one binary architecture.

3.4. Arithmetic and flags

The extended ALU implements 32-bit addition, subtraction, comparison, multiplication, division, remainders, bitwise operations, shifts, rotates, and signed and unsigned variants. `ADC` and `SBB` use the incoming carry or borrow; `MULHU` and `MULHS` return the upper half of the 64-bit product. `IMUL`, `IDIV`, and `IMOD` interpret operands as signed integers. `UMOD` and `DIV32` treat them as unsigned.

The condition flags follow the arithmetic rather than the register names. An addition sets `C` on unsigned carry and `V` on signed overflow. Subtraction sets `C` on unsigned borrow. Logical operations clear both. Shifts use their count modulo 32, with the last shifted-out bit becoming `C`. A zero count preserves the original value and the carry and overflow flags. Division by zero, and signed division overflow, take the divide-error exception.

```
LOAD32 R0, 0x7FFFFFFF
LOAD32 R1, 1
ADD32 R0, R1
BVS overflow
```

The addition produces `0x80000000` and sets `V`; the branch tests precisely that flag. Other branches distinguish signed and unsigned comparisons, so the same two register values may satisfy `BLT` and not `BLO`. The compiler uses these conditions when lowering the corresponding C comparison operators.

3.5. Control transfer and the stack

`JMP32` and `CALL32` use absolute 32-bit addresses, while `JMPR` and `CALLR` take their destination from a register. The relative forms give local control flow. Extended calls push a 32-bit return address and return through `RET32`. `PUSH32` and `POP32` move four-byte stack words. `MOVSP`, `SETSP`, and `ADDSP` allow compiled code to create a stack frame and recover it on return.

The 32-bit calling sequence is distinct from the original 16-bit stack instructions. The C compiler consistently uses `CALL32`, `RET32`, `PUSH32`, and `POP32`; it reserves the legacy forms for the earlier assembly programs. Nested function calls and recursion are therefore normal guest-machine operations. The stack-pointer register remains separate from the general registers.

3.6. Control registers and processor modes

Legacy mode is in force while the `EXT` bit of `CR0` is clear. Setting `EXT` activates the extended exception table and the privilege and address-translation machinery. In extended mode, supervisor execution can control the machine. User execution cannot change control registers, manipulate the TLB, halt the machine, or access privileged I/O instructions.

With `PROT` enabled and paging disabled, a user address below `CR6` is translated by adding the base in `CR5`; access beyond the limit takes a protection fault. With paging enabled, `CR3` selects the page tables and the address is interpreted as a virtual address. These choices belong to the processor's control state, not to the executable file format. A program can change modes using the appropriate privileged instructions while preserving the same instruction set.

3.7. Paging and address translation

The page size is 4096 bytes. The upper ten address bits select an entry in a page directory, the next ten select a page-table entry, and the final twelve are the byte offset. Page-directory entries refer to page tables; leaf entries give physical page frames and permission bits. The page-table flags are present (`P`), writable (`W`), user-accessible (`U`), executable (`X`), accessed (`A`), and dirty (`D`).

For a virtual address `va`, the indices and offset are:

```
pde = (va >> 22) & 0x3ff;
pte = (va >> 12) & 0x3ff;
off = va & 0xfff;
```

`pg_translate` reads the directory and table, checks the access, sets accessed and dirty bits, and returns the frame address with the page offset. A failed lookup raises vector 14. The fault address appears in `CR2`; the error code distinguishes a missing page from a permission violation, a read from a write, supervisor from user, and instruction fetch from data access. The software TLB stores cached translations. Writing the control state invalidates cached translations; `TLBFLUSH` removes them all and `INVLPG` removes the entry for one virtual page.

4. Instruction fetching and decoding

The emulator fetches instruction bytes through the same translation and physical-memory paths used for program data. `PC` identifies the first byte, and the decoder obtains the required number of bytes from the opcode's format. Before executing the instruction, the step function advances `PC` to the following instruction. A branch or call replaces that value; a fault records the original address so the instruction can be retried.

4.1. Instruction bytes and decoding

`decode` copies the encoded bytes of an instruction into a `DEnt`, which records their length. `fetch_byte`, `fetch_word`, and `fetch_dword` then read operands from that short byte sequence. The execution switch selects the opcode's operation. `src/dis.c` provides the textual form used by traces, the debugger, and the GDB interface. Disassembly does not change the machine's registers or the memory being inspected.

Instruction fetches are checked as executable accesses when paging is enabled. A 32-bit immediate may cross a page boundary; the fetch path handles that boundary through translation. The machine therefore applies page permissions to instructions as well as to

data. The instruction address stored in `cpu.inst_pc` is the point to which the processor returns after a recoverable fault.

4.2. Decoded-instruction cache

The emulator keeps decoded entries in RAM-page-sized tables. A previously decoded instruction at the same physical location reuses its bytes and length. When any write changes a RAM page, `cpu_mem_written` discards that page's decoded entries. Self-modifying code consequently fetches its replacement instructions on the next visit. The `-n` option disables this cache and executes through the uncached decoder.

The cache is an execution optimization, not a different machine mode. The `check_nocache` target repeats instruction-set, toolchain, kernel, GDB, and self-hosting tests with decoding disabled. Both modes produce the same guest output and state transitions. This separates the architectural behavior of a TORTOISE instruction from the host implementation used to decode it.

5. Machine state and memory

RAM occupies physical addresses from zero to the configured size minus one. The optional ROM occupies the top 64 KB of the 32-bit address space at `0xFFFF0000`; its bytes are read-only. Memory-mapped devices occupy the range beginning at `0xF0000000`. The emulator's physical access functions choose RAM, ROM, or a device by address and access width.

5.1. Physical memory accesses

`phys_read` and `phys_write` handle one-, two-, and four-byte accesses. RAM permits unaligned reads and writes. The device window accepts aligned 32-bit register accesses; a different width or alignment produces a bus error. Writes to the ROM are rejected. Every multi-byte RAM access is encoded in little-endian order, independently of the host C implementation.

The legacy video buffer is at `0x000B8000`, the original interrupt table begins at `0x00000100`, and the ordinary startup stack begins at the top of configured RAM. In extended mode the kernel decides which physical pages its process mappings expose. Physical devices do not become user-accessible merely because their numerical addresses fit in a program's 32-bit address range.

5.2. Virtual reads and writes

`mem_read` and `mem_write` operate on guest virtual addresses and use `translate` to obtain physical addresses. `vm_read` and `vm_write` also handle accesses that cross a page boundary by translating each affected byte. A normal aligned word whose physical bytes are contiguous uses a single physical access. These paths preserve the same visible result regardless of how the virtual pages are backed.

A read of a non-present page gives the kernel an opportunity to allocate it. A write to a read-only copy-on-write page similarly enters the page-fault handler. The access resumes at the saved `PC` after the kernel updates the mapping. Address translation is consequently the mechanism through which user programs obtain private memory, demand allocation, and process isolation.

6. Instruction execution

`execute_instruction` implements the defined opcodes in `src/cpu.c`. It reads registers and immediates from the decoded instruction, performs the operation, writes the destination register or memory, and updates condition flags where that instruction defines them. The call and return instructions modify both `SP` and `PC`. Privileged instructions inspect `U` before changing processor control state.

6.1. Results and memory operands

An instruction of the form `LDW R0, [R1 + 12]` computes an effective virtual address from its base register and signed displacement, translates that address, and reads a 32-bit word. `STB [R1 + 12], R0` stores only the low eight bits of the source. `LDBS` and `LDHS` extend the sign bit; `LDB` and `LDH` zero-extend. `LEA` computes an effective address without reading the addressed memory.

```
LOAD32 R1, 0x2000
LOAD32 R0, 0x12345678
STW [R1], R0
LDB R2, [R1 + 1]
```

The store puts four little-endian bytes at `0x2000`. The load reads `0x56` into `R2`. The architecture treats byte, halfword, and word operations as distinct instructions. The compiler selects them from the size and signedness of the C object being accessed.

6.2. Arithmetic instructions

The 32-bit arithmetic paths use helper routines for add and subtract carry rules, signed overflow, shifts and rotates, multiplication, and division. A binary arithmetic instruction updates its destination; `CMP32` and `TEST` set flags without preserving a result in the destination. `NEG` forms zero minus the operand. `RDCYC` exposes the low 32 bits of the machine's executed instruction count and `CPUID` reports the processor version value.

The emulator uses the fixed-width integer types of C to give these operations their 32-bit behavior. Signed division is handled with the signed interpretation of the stored bit patterns; unsigned division uses the full nonnegative 32-bit range. The result never depends on the word size of the computer hosting the emulator.

6.3. Branches and control flow

A conditional branch reads the current flags and selects its target if the predicate holds. `BEQ` and `BNE` read `Z`; `BLT`, `BGE`, `BLE`, and `BGT` combine `S`, `V`, and `Z` for signed relations. `BLO`, `BHS`, `BLS`, and `BHI` implement unsigned relations through `C` and `Z`. The other conditions test sign and overflow independently.

The C code generator uses these instructions for loops, if statements, short-circuit conditions, switches, and function returns. In the emulator, they are ordinary operations on the next-instruction `PC`. A jump that is not taken leaves `PC` pointing to the following instruction; a taken jump adds the signed displacement or installs the absolute target.

7. Deterministic execution and event scheduling

The machine measures simulated time in instructions executed. `cpu.icount` increases after each completed instruction, and devices schedule their future events against that counter.

Timer ticks and disk completion are therefore independent of the host's wall-clock scheduling. With deterministic clock mode selected, the real-time-clock device also derives seconds from the instruction count. An input file supplies a repeatable UART byte stream.

`step_one` first advances any device whose next event is due, then accepts an enabled pending interrupt, decodes one instruction, executes it, and increments the count. The timer's prescaler and period define interrupt intervals. A disk command completes after its sector count has consumed the specified instruction-based latency. A run limited by `-x` stops after the selected number of instructions with exit status 124.

`WFI` provides an efficient idle path. Rather than repeatedly executing a polling instruction, it advances the machine's instruction count to the next scheduled event that can raise an enabled interrupt. When that interrupt arrives, normal delivery resumes. The same mechanism serves the kernel idle loop and the assembly tests for waiting, timers, and interrupt masks.

8. Exceptions and system calls

The extended exception mechanism uses 64 handler addresses at the table selected by `CR1`. A fault saves the address of the faulting instruction, while a software trap saves the following instruction address. A fault during exception entry takes the double-fault vector, and a subsequent entry failure takes the machine's triple-fault stop path. These outcomes are explicit parts of the processor state transition.

8.1. Exception entry and return

`enter` constructs the saved machine frame. From user mode it switches to the stack address in `CR4` and saves the previous user `SP`. It then pushes the flags and return `PC`, records the vector and error code in `CR7`, clears the user and interrupt-enable state, and loads the handler address from the vector table. The `IRET` instruction restores the saved control-flow state and the original stack when returning to user mode.

```
LOAD32 R1, vectors
MTCR CR1, R1
STI
```

A kernel installs valid vector addresses before enabling interrupts. A page fault reports the inaccessible virtual address in `CR2` and the access classification in `CR7`. The kernel can allocate a missing page or terminate the user process with a signal. In either case it receives the original register state through the exception frame.

8.2. Software traps and kernel calls

`SYSCALL` enters the supervisor at vector 48. A guest process places the call number in `R0` and up to five arguments in `R1` through `R5`. The kernel dispatcher examines the trap frame, invokes the requested operation, and places its integer result back in `R0`. `SYSRET` and the extended interrupt-return path restore user execution under the same privilege rules.

`BRK` takes the breakpoint trap at vector 3. User software interrupts are allowed for vectors 48 through 63. A user attempt to execute `CLI`, `STI`, `HALT`, port I/O, control-register operations, a TLB operation, `WFI`, or a privileged return produces a protection fault at vector 13. These checks give the user and supervisor parts of the guest one consistent boundary.

9. Devices and memory-mapped I/O

`src/dev.c` defines the physical device bus. Each controller has a base address and a read/write callback. A 32-bit load or store to a device register invokes the corresponding callback rather than accessing RAM. The controllers are laid out at 4 KB intervals: interrupt controller `0xF0000000`, timer `0xF0001000`, UART `0xF0002000`, disk `0xF0003000`, keyboard `0xF0004000`, real-time clock `0xF0005000`, framebuffer `0xF0006000`, system control `0xF0007000`, and a second disk at `0xF0009000`.

The interrupt controller collects device lines in `PENDING`. `MASK` enables individual lines, `ACK` clears them, and `CLAIM` reports the lowest enabled pending line. The timer raises line 0, UART receive and transmit use lines 1 and 2, the primary disk uses line 3, the keyboard uses line 4, and the second disk uses line 6. The CPU converts an active line `n` into exception vector `32 + n`.

The UART writes transmitted bytes to the host's standard output. Input comes either from standard input or from the file selected by `-i`. The disk controller transfers blocks of 512 bytes between physical RAM and a disk image; `-D` selects a host file and `-E` supplies the second disk. `SECTOR`, `COUNT`, `ADDR`, `CMD`, `STATUS`, and `SIZE` make up the disk's register interface. The system-control device reports RAM size and machine version and exits with the value written to its power-off register.

Legacy port devices remain accessible through `IN` and `OUT`. Ports `0xFF00` through `0xFF04` supply console, timer, random-number, and halt functions; `0xFF10` through `0xFF13` expose a synchronous disk interface. The SDL build adds a 640-by-480, 32-bit framebuffer and host keyboard integration. In the normal terminal build, the UART and emulated disks supply the operating system's primary interaction path.

10. The TEX1 executable

TEX1 is the executable format used by the TORTOISE linker, ROM boot code, and operating-system process loader. It begins with the four-byte magic `TEX1` and a 32-byte header. The header contains an entry address, text address and size, data address and size, and BSS size. It is followed by the initialized text and data bytes. The remaining memory described by BSS is cleared by the loader.

A TEX1 image holds the addresses at which its instructions actually run. The host emulator can load one directly; the ROM and boot sector can read one from the disk; and the operating system can load one from a file. These paths share the same object representation even though they enter it through different loaders.

10.1. Sections and loading

The linker collects text sections from input objects at the chosen text base. Initialized data begins on the next 4 KB boundary, and BSS follows data. The loader reads text into the text address, copies initialized data to its address, clears BSS, and sets `PC` to the entry field. A directly loaded flat image instead starts at physical address zero, with unwritten RAM filled with the `HALT` opcode `0xFF`.

```
./as -c first.s -o first.o
./as -c second.s -o second.o
./tlink -T 0x400000 -e _start -o program first.o second.o
./tortoise -m 16 program
```

In this example the assembler produces relocatable input files. `tlink` assigns final addresses and writes a TEX1 header. The emulator's `load_tex` uses that header rather than assuming that code begins at address zero. The operating system uses the same format for processes whose user text normally begins at `0x00400000`.

10.2. Relocations and symbol resolution

A TOF object may refer to a symbol defined in another object. The assembler records the section, location, relocation type, symbol, and addend. The linker concatenates each kind of section, computes global addresses, and patches the references. Its relocation kinds are 32-bit absolute (`R_ABS32`), 32-bit PC-relative (`R_REL32`), and 16-bit absolute (`R_ABS16`).

For a 32-bit relative displacement, the patched value identifies the target in relation to the following instruction:

```
rel = target - (field_address + 4);
```

A global symbol table resolves exported names across all input objects. The linker also defines `_etext`, `_edata`, and `_end`, which programs use to find section boundaries. An entry symbol is chosen with `-e`, from `_start`, or from the beginning of text. The `-M` option prints the resulting address map. Thus a C function compiled in one file can call a function in another without their locations being known at compile time.

11. Bare-machine programs

A bare-machine program can be assembled into a flat binary or compiled from C with `-bare`. Bare code does not make Unix system calls or assume that the operating system has established user-mode mappings. It uses the machine's registers, RAM, UART, and system-control device directly. The bare C startup sets `SP` to the upper end of RAM, clears BSS, calls `main`, and terminates by writing the exit status to the power-off register.

```
./tcc0 -bare tests/c/sieve.c -o sieve
./tortoise -m 16 sieve
```

The `-bare` option selects link address zero and the bare runtime. Normal compilation selects the operating-system address `0x00400000` and the system-call runtime. The source-language parser and code generator are the same in both modes. The distinction is the memory layout and the implementation of library and startup functions.

Assembly programs under `assembler/` exercise bare execution through arithmetic, calls, the legacy port interface, privilege checks, timer events, direct memory, paging, and I/O devices. This path is also how the ROM and the disk boot sector are built: they use TORTOISE instructions without depending on a preexisting TORTOISE operating system.

12. ROM and boot images

The normal ROM image is assembled from `rom/rom.asm` at address `0xFFFF0000`. A ROM boot begins with `PC` at that address and `SP` established from the system-control RAM-size register. The ROM prints a banner over the UART, examines the first sector of disk 0, and enters either the disk bootstrap or its built-in monitor. The entire boot path consists of machine instructions interpreted by the emulator.

12.1. Disk layout

`tdisk` creates the bootable image. Sector zero contains 512 bytes of boot assembly. The linked kernel TEX1 image starts at sector one, and the TFS file system begins at sector 1024. Both the boot sector and the file-system offset are recorded by the image layout rather than discovered through a host operating-system filesystem API.

```
make os
./tortoise -m 16 -r rom/rom.bin -D disk.img
```

When the ROM finds the sector-ending signature `0x55 0xAA`, it loads the sector at physical address `0x00010000` and jumps there, passing the installed RAM size in `R0`. When no boot signature is present, the ROM runs a monitor that can read memory, write bytes, jump to an address, retry a boot, and report RAM size. The commands `m`, `w`, `g`, `b`, and `r` operate directly on the guest machine.

12.2. The disk boot sector

`os/boot/boot.asm` uses the disk's memory-mapped register interface. It reads the TEX1 header from the image beginning at sector one, determines the number of sectors containing the kernel, and loads them into a temporary buffer at `0x00200000`. It copies the text bytes to their linked text address, copies the data bytes to their linked data address, and clears the kernel's BSS range.

A disk command writes the sector, count, physical destination, and read command into the controller. The boot code polls the status register until the transfer is complete. The image's format makes the kernel entry and section sizes available before C code is entered. On an invalid executable the boot sector writes an error character to the UART and stops. Normal startup follows the entry address in the TEX1 header.

12.3. Kernel entry

The kernel is linked at `0x00020000` with `_start` as its entry. The assembly entry in `os/kernel/entry.s` disables interrupts, establishes the kernel stack from `bootstack_top`, passes the RAM size to `kmain`, and enters the C kernel. That C function installs its exception table, initializes physical memory and paging, prepares disk and mounted file systems, creates the first process, starts the timer and interrupt controller, and enters the scheduler.

The boot code and kernel run in supervisor mode. Paging is enabled after the initial memory structures exist. The kernel then creates user page directories and changes into user mode by returning through an appropriate trap frame. The boot sequence crosses from immutable ROM to disk code, from disk code to linked kernel assembly, and from kernel assembly to C, all on the TORTOISE instruction set.

13. Privileged instructions and interrupts

The CPU exposes its protected operations as ordinary opcodes rather than as host function calls. A kernel reads and writes control registers, installs vector tables, enables the device interrupt mask, acknowledges interrupts, flushes translations, and waits for an interrupt. Each operation is executed by the emulator with the privilege rules of the emulated CPU.

13.1. Control-register operations

`MFCR` reads a control register into a general register and `MTCR` changes a writable control register. Writes to `CR0` or `CR3` invalidate translated-address caches. `TLBFLUSH` discards all translations, and `INVLPG` discards one virtual page's translation. The page-fault handler uses these operations when a process gains a physical page or when a copy-on-write mapping changes its permissions.

```
LOAD32 R0, page_directory
MTCR CR3, R0
LOAD32 R0, 5
MTCR CR0, R0
```

The control value in the example enables extended mode and paging. The processor will subsequently translate user and supervisor addresses through the directory supplied to `CR3`. A kernel can enter and leave address spaces without rebuilding the emulator: it changes the register containing the page-directory base.

13.2. Interrupt and exception handlers

The assembly stub `alltraps` saves `R0` through `R7`, passes a pointer to the saved register frame to the C `trap` routine, and later restores the registers before `IRET`. The same frame represents a timer interrupt, a system call, and a CPU fault. `trapret` is the common exit path. `forkret` restores a newly created process's frame, and `first_start` enters the first user process through the same machinery.

The CPU reports a timer on vector 32 and a system call on vector 48. A page fault arrives at vector 14 and an invalid instruction at vector 4. The kernel distinguishes them by the saved cause. A timer handler accounts for running time, updates timers, acknowledges the controller, and invokes scheduling when needed. A system-call handler reads the requested service and its arguments from the saved registers. The exception return restores the interrupted context without exposing host registers to the guest.

13.3. Interrupt controllers and devices

A device signals its line through `pic_raise`, which updates the controller's pending bits. At the beginning of an instruction step, the CPU checks the mask and interrupt-enable flag, selects the lowest enabled pending line, and enters vector `32 + n`. The kernel acknowledges the line by writing to the PIC's `ACK` register. Unacknowledged lines remain pending.

The timer uses `COUNT`, `PERIOD`, `CTRL`, and `PRESCALE`. The kernel writes a prescale of 1000 and a period of 10, which produces a kernel tick every 10000 emulated instructions. The UART signals receive readiness when enabled. Disk operations raise an interrupt at their completion event. The default kernel can perform console and file-system work without an SDL display because those services use the UART, timer, disk, and control-register devices.

14. Kernel facilities

The kernel is a set of C translation units supported by a small amount of assembly. `main.c` starts the system; `mm.c` manages physical and virtual memory; `kmem.c` serves kernel allocations; `proc.c` manages processes; `trap.c` dispatches traps and system calls; `signal.c` delivers signals; `file.c` implements descriptor operations; `vfs.c`, `tfs.c`, `ramfs.c`, `devfs.c`,

and `procfs.c` implement file systems. `tty.c` handles the terminal. The kernel exports its definitions through `os/kernel/kernel.h`.

The assembly functions in `entry.s` provide the boundary that C cannot describe directly: startup, trap save and restore, context switching, control-register moves, portless device reads and writes, and the user-mode transition. `memops.s` supplies low-level memory operations. The remaining kernel code uses normal C functions, pointers, structures, arrays, and the calling convention generated by `tcc0`.

The kernel owns physical resources and exposes processes and files through system calls. It uses supervisor-only mappings for low physical memory and devices. Every process has a page directory, a kernel stack, a saved execution context, descriptors, a current working directory, a parent, and signal state. The system can schedule up to 64 processes and keep up to 32 descriptors per process.

15. Building the TORTOISE kernel

The kernel is compiled by TORTOISE's own C compiler. The Makefile invokes `./tcc0 -c` for each kernel `.c` file and `./as -c` for its assembly. `tlink` joins these TOF objects at text address `0x20000` with `_start` as the entry symbol. The result is `os/kernel/kernel.tex`. This is a linked guest executable, not host x86 machine code.

```
./tcc0 -c os/kernel/mm.c -o os/kernel/mm.o
./as -c os/kernel/entry.s -o os/kernel/entry.o
./tlink -T 0x20000 -e _start -o os/kernel/kernel.tex \
  os/kernel/entry.o os/kernel/memops.o os/kernel/*.o
```

The Makefile supplies the complete kernel object set for the real link, including the memory, file-system, signal, process, trap, and device modules. `tdisk` inserts the resulting TEX1 kernel immediately after the boot sector. The same image also contains executable user programs compiled by `tcc0` and the files under `/etc` required by the startup process.

The booted kernel first installs a 64-entry exception table and then initializes its page allocator and virtual mappings. It attaches the initial disk and the root file system, initializes console and process state, programs the timer, enables the interrupt lines, and enters the scheduler. When `/etc/kernel.conf` requests `selftest`, it also runs its kernel-heap test during this initialization sequence.

16. Processes, scheduling, and signals

A `Proc` represents a guest process independently of the host process running `tortoise`. The kernel manages the process's user address space, saved registers, kernel stack, files, signal dispositions, identity, process group, session, parent, and accounting counters. The scheduler saves and restores execution contexts with `swtch`, whose assembly preserves the callee-saved registers and exchanges stack pointers.

The runnable queue is selected by the lowest nice value, from -20 to 19. Processes of equal priority run in rotation. A quantum is expressed in kernel ticks, and a newly runnable higher-priority process can preempt the current one. Processes waiting for disk, pipe data, terminal input, a child, or a timed event sleep on wait queues; the timer awakens timed sleepers. When nothing is runnable, the idle loop enters `WFI` rather than consuming a busy loop of simulated instructions.

16.1. The first user process

`user_init` constructs process 1 and its initial context. `init_entry` enters `/bin/init`, which processes `/etc/inittab` and starts the shell. The system's normal interactive interface is therefore a user process reading and writing file descriptors, not a loop inside the emulator's host debugger. The shell uses `fork`, `exec`, `wait`, `pipe`, and file-descriptor redirection to run other executables.

`execve` replaces the calling process's address space with a linked TEX1 program. Its text and data are mapped at the addresses recorded in the image. The kernel constructs arguments, environment strings, and the initial user stack. On entry, `R1` carries `argc`, `R2` carries `argv`, and `R3` carries the environment pointer. The runtime startup saves `environ`, calls `main`, and passes the return status to `exit`.

The loader also recognizes a `#!` interpreter line. It runs the named interpreter with the script name and remaining arguments. Descriptor close-on-exec state, ignored and caught signals, and process identity are handled by the exec path. A child that exits becomes waitable, sends `SIGCHLD` to its parent, and is collected with `wait` or `waitpid`.

16.2. Fork, scheduling, and signals

`fork` constructs a new process and duplicates the parent's execution state without copying all its user-memory bytes. Parent and child initially refer to the same physical pages; private writable pages are made read-only and marked copy-on-write. On a later write, the page-fault handler allocates a private copy when the page is still shared. This makes process creation depend on page references rather than the size of the process's memory image.

Signals have pending and blocked masks and per-signal handlers. The kernel delivers a pending unblocked signal before returning to user mode. A caught signal gets a user-stack frame containing saved registers and mask state; the library restorer calls `sigreturn` to recover them. `SIGKILL` and `SIGSTOP` cannot be caught or blocked. Terminal-generated interrupts and job-control stop signals act on process groups, and an orphaned stopped group receives the appropriate hangup and continue behavior.

A user page fault or protection fault becomes `SIGSEGV`; invalid instructions, divide errors, bus errors, and breakpoints become `SIGILL`, `SIGFPE`, `SIGBUS`, and `SIGTRAP`. A caught handler can repair the cause and return to retry the original instruction. Timer-driven scheduling, system-call suspension, signal delivery, and page-fault recovery consequently pass through the same saved trap frame and user-return path.

17. The file system and userspace

The kernel implements one file interface over several backing file systems. Vnodes refer to files or directories and contain a reference to their mounted filesystem. Open descriptors, current directories, and mount points retain vnodes while they are in use. Paths are resolved component by component, including `.` and `..`, symbolic links, and transitions across mount points.

17.1. The virtual file system

`vfs.c` defines the shared lookup and mount operations. Filesystem implementations provide methods for lookup, create, read, write, directory enumeration, metadata, links, rename, truncate, I/O control, and polling. `file.c` translates system calls on descriptors into these

methods and implements pipes, descriptor duplication, paths, access checks, and seek state. An unlinked open file remains usable until its final descriptor closes.

At boot the root is the disk-backed TFS filesystem. `devfs` is mounted at `/dev`, `procfs` at `/proc`, and an in-memory filesystem at `/tmp`. The mount system also accepts another TFS disk, including the second disk exposed by `-E`. A mount can be removed when its vnodes are no longer in use. These operations give the shell a directory tree independent of the physical type of the data beneath it.

17.2. TFS and disk storage

TFS stores data in 512-byte blocks. Block zero contains the `TFS2` superblock and its layout and free-space counters. An inode table, block bitmap, and data blocks follow. The filesystem has 8192 inodes of 128 bytes each. An inode holds ownership, mode, link count, file size, timestamps, twelve direct block addresses, and single, double, and triple indirect block addresses. Directories contain fixed 32-byte name-and-inode records. The root has inode number one.

`os/kernel/tfs.c` implements inode allocation, block allocation, file read and write, directory lookup, hard links, symbolic links, rename, truncate, and recursive release of indirect blocks. It uses a 64-block cache and writes through to the disk. The host utility `tdisk` creates the on-disk representation and can also inspect or extract its files. It writes TFS2 and reads both TFS2 and the original TFS1 format.

```
./tdisk -o disk.img -b os/boot/boot.bin \
  -k os/kernel/kernel.tex os/bin/sh:/bin/sh
./tdisk -l disk.img
./tdisk -x disk.img /bin/sh sh.copy
```

`ramfs` stores files in guest memory. `devfs` exposes `/dev/console`, `/dev/tty`, `/dev/null`, `/dev/zero`, `/dev/random`, `/dev/urandom`, `/dev/kmsg`, two disks, and the graphical and keyboard devices. `procfs` exposes process and system information. Each uses the VFS interface rather than requiring a separate system-call family.

17.3. The shell and terminal

`os/bin/sh.c` implements the shell. It reads command lines, keeps double-quoted strings together, separates pipe and redirection operators, and executes commands by forking and replacing child processes. It recognizes input redirection `<`, output redirection `>`, append `>>`, pipelines `|`, and trailing `&` for background execution. It performs `cd` and `exit` in the shell process and executes the other programs through their paths or through `/bin`.

The TTY layer handles line editing, echo, input and output processing, signal-generating control characters, and foreground process groups. A canonical read collects input until the end of a line, while termios settings permit other character-handling modes. Console descriptors initially belong to init and are inherited by its children. Terminal I/O flows through the UART device, its interrupt line, a TTY queue, file descriptors, and finally the shell.

The command directory contains `init`, `sh`, `echo`, `cat`, `ls`, `mkdir`, `rm`, `wc`, `hello`, `ps`, `kill`, `sleep`, `shutdown`, `crash`, `mount`, `umount`, and `getty`. The tests run these programs against ordinary files, pipes, terminal state, mounted devices, and concurrently executing processes. The

system accepts commands through a real user-mode shell whose executables were produced by its own toolchain.

18. Multiple source files

The assembler can produce a TOF object from each assembly input. The C compiler uses the same object format for each translation unit. Every object contains its section sizes, section data, symbols, and relocations. Compilation does not require the source defining a called function to be read at the same time as the source calling it. A declaration supplies the function type; a relocation supplies the eventual address.

```
/* square.c */
int square(int n) { return n * n; }

/* main.c */
int square(int);
int main(void) { return square(7) == 49 ? 0 : 1; }
```

The two files can be compiled and linked separately:

```
./tcc0 -c square.c -o square.o
./tcc0 -c main.c -o main.o
./tlink -T 0x400000 -o program main.o square.o lib/os/crt0.o \
  lib/os/sys.o lib/os/stdlib.o lib/os/stdio.o \
  lib/os/string.o lib/os/ctype.o lib/os/memops.o
```

In an ordinary user compilation the `tcc0` driver supplies the runtime objects and linker options automatically. The explicit link above illustrates how symbol resolution joins generated objects with library code. The same separate-compilation scheme builds the kernel, which consists of C modules and assembly files sharing definitions from `kernel.h`.

19. Assembling TORTOISE instructions

The assembler is a two-pass program implemented in `assembler/assembler.c`. It reads source lines, expands included files and macros, parses labels and expressions, assigns sizes and addresses, and emits the selected instruction encodings. It uses an instruction table containing mnemonic, operand class, and opcode. Upper- and lower-case register and instruction spellings denote the same operation; labels preserve their original case.

An operand can be a register, `SP`, a control register, a memory reference, or an expression. Expression parsing recognizes decimal, hexadecimal and binary integers, character constants, labels, current location `.`, parentheses, and the usual integer arithmetic and bitwise operators. Forward labels are resolved on the later pass. Errors carry a source filename and line number.

```
.globl _start
.text
_start:
    LOAD32 R0, 20
    LOAD32 R1, 22
    ADD32 R0, R1
    HALT
```

A flat assembly run begins at physical address zero unless its initial origin selects a high fixed address, as the ROM does. Object mode instead writes section-relative offsets that can be relocated by the linker. The instruction-selection and expression rules are common to both modes.

19.1. Sections, directives, and locations

The assembler recognizes `.text`, `.data`, and `.bss` for object sections. `.globl` exports a symbol. `.byte`, `.word`, and `.dword` emit one-, two-, and four-byte integers; `.ascii` and `.asciiz` emit string bytes, with the latter appending zero. `.space` reserves or fills bytes; `.align` adjusts the location; `.equ` defines a constant; `.include` reads another source. `.org` sets the origin used for flat-image placement.

```
.data
message: .asciiz "ready"
.align 4
counter: .dword 0
.text
.globl _start
_start:
    LEAPC R0, message
    HALT
```

For flat images, `.org` normally fills the gap from earlier output to the requested address. When the initial origin is at least `0x10000000`, the image instead records that address as its starting location. This permits the ROM to be assembled for `0xFFFF0000` without writing a nearly four-gigabyte file. In TOF mode, text, data, and BSS have their own offsets, with unresolved symbol references retained as relocations.

19.2. Assembler macros and local labels

A `.macro` definition names parameters and stores body lines until `.endm`. Macro invocation substitutes parameters into the lines that will be assembled. The special expansion number distinguishes labels generated by separate invocations. A label beginning with a dot is local to the preceding nonlocal label, which lets a function contain short loop and branch names without colliding with another function's labels.

```
.macro save reg
    PUSH32 \reg
.endm

.globl _start
_start:
    LOAD32 R0, 7
    save R0
    POP32 R1
    HALT
```

The expansion is processed before instruction emission. The final code is identical to the instructions written out by hand. The tests cover includes, expressions, directives, local and forward labels, instructions of different lengths, macro expansion, and references patched after the target's address becomes known.

20. The C compiler

`tcc0` is a C compiler targeting TORTOISE rather than the host instruction set. `cc/pp.c` reads and preprocesses source; `cc/parse.c` constructs types, symbols, declarations, expressions, and statements; `cc/gen.c` writes the corresponding assembly; and `cc/main.c` arranges compilation, assembly, and linking. This separation also makes `-S` useful: it writes the actual TORTOISE assembly before the assembler turns it into bytes.

The parser builds `Node` trees for operators and statements, `Type` records for scalar and aggregate types, `Sym` entries for names and storage, `Func` records for function bodies, `GVar` records for globals, and `Init` lists for initial data. `parse_program` reads one translation unit into these records. `gen_program` emits its functions and global objects as `.text`, `.data`, and `.bss` assembly. These intermediate structures describe the program in the target's 32-bit type system, not in the host's C ABI.

```
./tcc0 -S tests/c/arith.c -o arith.s
./as -c arith.s -o arith.o
./tcc0 tests/c/arith.c -o arith
```

The generator uses `R0` for expression values and computed addresses, moves additional operands through `R1`, and uses the stack for intermediates whose evaluation can overwrite registers. Typed loads distinguish signed and unsigned byte or halfword objects from word objects. Assignments and structure copies use the size determined by the parser. Conditionals and loops become branches to numbered assembly labels.

A function starts with a prologue saving `R7`, copies `SP` into `R7`, and subtracts its required local-frame size. It ends by restoring the stack and frame pointer and executing `RET32`. The compiler uses absolute or relative call instructions according to the generated expression. Functions and globals are emitted with C-compatible symbol names, and internal names remain local to their translation units.

21. TOF objects and relocations

The relocatable format begins with the magic `TOF1`. Its header records text, data, and BSS sizes, counts of symbols and relocations, and the length of a string table. The file then stores the initialized text and data bytes, fixed-size symbol records, relocation records, and symbol strings. BSS consumes addresses but no initialized bytes. `as -c` writes this format and `tlink` reads it.

Each symbol records a name-table offset, a value relative to its section, a section classification, and linkage information. Relocations record the patch offset, section, kind, referenced symbol, and addend. Assembler labels exported with `.globl` appear in the linker-visible symbol namespace. Local labels remain associated with the object that defines them.

21.1. Global symbols and patching

`tlink` loads the objects, assigns their text and data section bases, and builds a hash table of global names. A defined global supplies an address. A reference from another object is matched to that entry. Absolute relocations write the target address plus addend; relative relocations write a displacement measured from the relocation field's end. The 16-bit form writes only the required two bytes.

The linker reports multiply defined global symbols and unresolved references. It also defines its section-boundary symbols after layout, so a bare allocator or runtime can begin at `_end` without a hard-coded heap address. This is the same mechanism that lets the ROM, kernel entry, system-call library, and C-generated functions be written and built in separate files.

22. The linker

`tools/tlink.c` is the complete guest executable linker. Its inputs are TOF objects, and its output is either a TEX1 executable or a flat binary. `-T` chooses the text address, `-e` chooses the entry name, `-o` selects the destination, `-M` prints a symbol map, and `-b` selects flat output. The default text address is zero. With no explicit entry, `_start` is used when defined.

The linker first determines total section sizes and the placements of individual object sections. It places text at the requested base, aligns data to the next 4 KB boundary, and places BSS after initialized data. It then assigns each symbol its final address. The relocation pass patches text and data before the writer emits the executable header and initialized bytes.

```
./tlink -T 0x20000 -e _start -M \
  -o os/kernel/kernel.tex os/kernel/entry.o os/kernel/main.o
```

The Makefile's actual kernel link supplies every kernel object. The command illustrates the relevant switches and their effect on layout. In normal operating-system builds `tlink` establishes the addresses used by boot-sector copying, kernel entry, user-program loading, and pointers stored in initialized global objects.

23. The compiler driver

The compiler driver accepts `.c`, `.s`, `.asm`, and `.o` inputs. For a C file it forks a compilation worker, runs preprocessing and parsing, and writes assembly. Unless `-S` was requested, it invokes the assembler for a TOF object. Unless `-c` was requested, it invokes `tlink` over all accumulated objects and the runtime library. It finds its tools alongside its own executable and locates library headers and objects relative to that location.

`-I` adds include directories, `-D` supplies preprocessor definitions, `-o` names the output, and `-keep` retains intermediate files. `-bare` defines `__BARE__` and selects bare startup and library objects, with text linked at zero. Ordinary compilation selects system startup and library objects and text at `0x00400000`. Thus the same driver can build a bare test, a userspace application, or any C module assembled for the guest.

```
./tcc0 -c part.c -o part.o
./tcc0 -S part.c -o part.s
./tcc0 part.c helper.c -o program
./tcc0 -bare part.c -o bare-program
```

Each source is a separate translation unit. The driver links their assembled objects after compilation completes. The `make` targets use precisely this arrangement for the standard library, the C tools being bootstrapped, all kernel modules, and each user command.

24. Object formats and the C runtime

TORTOISE uses three distinct output representations. A flat binary is a direct sequence of machine-code and data bytes placed at a known address. A TOF file is a relocatable

build artifact containing sections, symbols, and relocation records. A TEX1 file is a linked executable with a header carrying runtime addresses and an entry point. Their roles correspond to boot code, separately compiled modules, and programs that can be loaded from a filesystem.

24.1. Bare and hosted executables

A bare executable is linked at address zero and uses `lib/crt0_bare.s` with the device-backed library in `lib/bare`. It initializes a machine without kernel services. A hosted executable is linked at `0x00400000` and uses `lib/crt0.s` and objects in `lib/os`. It enters through the operating-system loader, receives argument and environment pointers, and uses a system call to access files, processes, time, and other services.

The same sources implement strings, memory allocation, character operations, formatted output, and portions of the standard C library in both builds. The differing `sys_bare.c` and `sys.c` supply services appropriate to their environment. That division makes it possible to run the compiler and the library inside the emulated operating system while also using the library in a program that executes on the raw emulated processor.

24.2. Startup and library calls

The hosted startup stores the initial environment, calls `main`, and calls `exit` with its return value. `lib/sys.c` issues the guest `SYSCALL` instruction with the correct system-call number and arguments. Negative kernel results become the C convention of `-1` with `errno` set. The library covers `open`, `read`, `write`, `close`, `fork`, `execve`, `waitpid`, pipes, directories, `stat`, `mmap`, `munmap`, signals, clocks, terminal control, and file-system operations.

`lib/stdlib.c` implements dynamic allocation through a free list, integer conversion, sorting, pseudorandom numbers, and environment operations. `lib/stdio.c` implements file streams and formatted input and output, including integer formatting, flags, width, and precision. `lib/string.c` supplies string routines, and `lib/memops.s` supplies memory operations in TORTOISE assembly. These functions are ordinary linked guest functions. The compiler, shell, linker, and user utilities call them by the same convention.

24.3. Calling conventions

The ABI places arguments on the stack, each in a four-byte slot, in reverse order. The caller removes the argument area after the call. `R0` through `R3` are caller-saved, `R4` through `R6` are callee-saved, and `R7` is the frame pointer. An ordinary C function saves `R7`, sets it to the incoming `SP`, allocates locals below it, and reads the first normal argument at `[R7 + 8]`. Its return address is at `[R7 + 4]` and its result is placed in `R0`.

```
.globl add
add:
    PUSH32 R7
    MOVSP R7
    LDW R0, [R7 + 8]
    LDW R1, [R7 + 12]
    ADD32 R0, R1
    SETSP R7
    POP32 R7
    RET32
```

This function has the same calling interface as `int add(int, int)` compiled by `tcc0`. Structure arguments are copied into caller-created stack slots. A function returning a structure receives a hidden pointer to the destination as its first argument. Variadic functions use the position of the first unnamed stack argument, and `<stdarg.h>` traverses four-byte argument slots. The assembler and C compiler use the identical ABI, allowing the kernel's assembly stubs to call its C routines.

25. Language implementation

The C language accepted by `tcc0` covers `char`, `short`, `int`, and `long`, signed and unsigned forms, pointers, arrays, functions and function pointers, structures, unions, enumerations, typedef names, global and local objects, initializers, and the C statements and expression operators used by the source tree. An `int` and a `long` are each 32 bits on this machine. A pointer is four bytes. There is no dependency on the host's 64-bit pointer representation when a guest program is compiled.

The compiler handles static and external linkage, recursive calls, casts, address and dereference operations, short-circuit logical expressions, conditional expressions, loops, switches, labels, `goto`, and variadic functions. It reports source errors using the token's filename and line. The project source is also a direct use of this language subset: `tcc0` compiles its own four C modules and the assembler and linker C implementations into runnable guest programs.

25.1. Types and declarators

`Type` records the kind, signedness, size, alignment, base type, array length, members, function parameters, and variadic or prototype status. `ptr_to`, `array_of`, and `func_type` construct derived types. A declarator can combine pointer stars, brackets, and parentheses so that an array of pointers differs from a pointer to an array. Typedef names resolve to the same type records used by ordinary declarations.

```
int values[8];
int *table[4];
int (*callback)(int);
struct Pair { int x, y; };
```

These declarations cause the parser to compute different object sizes and levels of indirection. A structure member has a byte offset within its containing object. A union's members share the same starting offset. The resulting type and offset information is attached to expression nodes and used when `gen.c` emits effective-address calculations or loads and stores.

25.2. Lvalues and conversions

The generator distinguishes an object's address from its loaded value. `gen_addr` emits an address in `R0`; `load` selects `LDB`, `LDBS`, `LDH`, `LDHS`, or `LDW` from the object's type. An assignment evaluates its destination address and source value, then selects `STB`, `STH`, or `STW`. Aggregate assignments copy the declared number of bytes through memory rather than treating a structure as a single register.

```
unsigned char byte = 255;
int x = byte;
short y = (short)x;
```

The load of `byte` zero-extends it, so `x` receives 255. Conversion to a signed short keeps the value representable in 16 bits. Pointer arithmetic multiplies an integer offset by the size of the referenced object, so incrementing a pointer to a structure advances by that structure's size rather than by a single byte.

25.3. Expressions and control flow

`cc/parse.c` creates expression nodes for arithmetic, comparison, assignment, calls, casts, increments and decrements, short-circuit operations, conditionals, and comma expressions. The generator visits the expression tree and emits instructions in a defined evaluation order. It uses immediate or local-variable fast paths when the second operand can be read without a nested evaluation. For more complex expressions it saves intermediate values on the stack.

`gen_branch` recognizes comparison and logical nodes and emits signed or unsigned conditional branches as required. `gen_stmt` lowers blocks, declarations, `if`, `while`, `do`, `for`, `switch`, `break`, `continue`, `goto`, and `return` into control-flow labels. The compiler emits the function's common epilogue once and directs returns to it. Structured C control flow thus becomes jumps, tests, stack operations, and register assignments in the emulated instruction set.

25.4. Preprocessing and includes

`cc/pp.c` tokenizes source while retaining file, line, whitespace, and beginning-of-line information. It recognizes the two C comment forms, string and character literals, numbers, identifiers, punctuators, and backslash escapes. A token's source location follows it through macro expansion and is used for diagnostics.

Quoted includes are resolved relative to their source and the include search paths. The preprocessor recognizes `#include`, `#define`, `#undef`, `#if`, `#ifdef`, `#ifndef`, `#elif`, `#else`, `#endif`, `defined`, and `#error`. Conditional expressions are evaluated before parsing C declarations. Only tokens in active source groups enter the C parser. Include paths supplied with `-I` and names supplied with `-D` use that same preprocessor machinery.

25.5. Function-like macros

A function-like macro has a list of formal parameters and a replacement token sequence. Expansion substitutes actual arguments, expands nested tokens, and implements stringification with `#` and token concatenation with `##`. The preprocessor tracks macro-expansion state so that a macro does not expand itself indefinitely while it is active.

```
#define JOIN(a, b) a ## b
#define STR(x) #x
#define TWICE(x) ((x) + (x))
int JOIN(value, _count) = TWICE(4);
```

After preprocessing the declaration defines `value_count` initialized to eight. The compiler does not need a special code-generation case for the macro: the parser receives the tokens of the resulting C declaration. The self-hosted compiler uses these mechanisms while compiling its own headers and implementation.

25.6. Global initializers and strings

Global declarations are represented by `GVar` records. Initialized objects are emitted into `.data` with `.byte`, `.word`, `.dword`, and reserved gaps for zero bytes. A global whose initializer

references another symbol emits a symbolic `.dword` entry, which becomes a TOF relocation after assembly. A tentative or zero-initialized global is placed in `.bss`.

Array, structure, union, and string initializers are lowered into offset-specific initializer records. A string literal receives an internal label and its own byte sequence. For automatic aggregates, the generator clears storage and emits the stores needed for initialized fields. The resulting assembly uses ordinary memory operations and the section-directive machinery shared with hand-written assembly.

25.7. Variadic functions

A variadic function receives named parameters at fixed stack offsets and additional arguments in the following stack slots. The C preprocessor and parser recognize the variadic function declaration and built-in forms used by `<stdarg.h>`. `va_start`, `va_arg`, and the associated state traverse the caller's argument area according to the same four-byte stack convention used by regular calls.

`printf` and its relatives exercise this path directly. Integer and pointer arguments pass through the guest stack, and the library's formatted-output routines retrieve them with the guest ABI. This also tests the agreement between separately compiled callers and callees: the compiler that produces a user program and the compiler that built the library must interpret exactly the same stack frame.

26. Per-process virtual memory

`os/kernel/mm.c` manages the physical frames backing every process. A low-memory allocator serves page directories, page tables, kernel stacks, and the kernel heap below 4 MB. Memory above that boundary is tracked by a bitmap and per-page reference counts. The kernel heap divides smaller pages into allocation classes and serves larger requests with whole pages.

Each user process has a page directory. Supervisor-only low-memory mappings and device mappings are shared, while user text, data, heap, stack, and memory-mapped regions belong to that process. A new executable begins at `0x00400000`. The user stack ends near `0x7FFFF000` and can grow as needed up to its configured one-megabyte limit. `sbrk` extends the heap, and anonymous `mmap` regions are placed downward from `0x60000000`, unless a fixed address was requested.

`fault_in` allocates missing demand-zero pages and handles writes to copy-on-write pages. `fork_space` shares existing physical pages, marks private writable mappings with the COW bit, and increments their references. A later write creates a private copy when required. `do_mmap` and `do_munmap` add, remove, or split process regions. Shared anonymous mappings retain shared backing across `fork`. `user_ok`, `copyin`, `copyout`, and `copyinstr` validate and transfer data at the user/kernel boundary.

The address translation and fault mechanism make these operations visible to software as ordinary memory. A C process can call `malloc`, touch a newly extended heap page, fork a child, or write to a private mapping without receiving a host allocation object. Its instructions read and write guest virtual addresses, and the kernel chooses the physical memory backing those addresses.

27. Self compilation and testing

The project tests the machine, assembler, linker, compiler, C library, ROM, operating system, and complete self-hosting path. The top-level `make check` runs CPU tests, legacy output comparisons, extended assembly tests, device and UART tests, ROM tests, linker tests, C programs and compile-error tests, disk-image tests, operating-system tests, GDB tests, and the self-hosting test. The tests exercise the same executable paths used by the interactive system.

```
make check
make check_nocache
make run_os
make run_selfhost_test
```

The legacy check assembles original programs and compares their binaries and output with frozen baseline files. Extended tests exercise 32-bit operations, branch conditions, flag handling, paging, protection, DMA, interrupt control, and waiting for device events. The unit suites test instruction execution and device behavior directly. The C tests cover arithmetic, pointers, globals, structures, recursion, function pointers, variadic arguments, strings, sorting, allocation, preprocessing, and an interpreter implemented in C.

The operating-system suite boots fresh disk images and compares the console output with expected transcripts. Its cases cover process creation, copy-on-write, exec, signals, lazy allocation, `mmap`, terminal behavior, file operations, larger files, polling, and devices. It also constructs second-disk images for tests that use disk 1. The tests use a deterministic clock, scripted UART input, and an instruction limit. This supplies a reproducible external observation of the whole guest system.

The self-hosting image contains the assembler, linker, compiler, runtime libraries, headers, and a C source file at `/src/hello.c`. These tools were compiled for TORTOISE by `tcc0` on the host. Once the image boots, the guest compiler compiles that source, the guest assembler produces its object, the guest linker creates its executable, and the guest runs it. The test extracts `/home/hello.s` from the resulting disk and compares it byte for byte with the assembly produced by the host invocation of `tcc0 -S` on the same source. The comparison succeeds.

```
./tcc0 -S tests/selfhost/hello.c -o hello-host.s
# Inside the booted TORTOISE system:
tcc0 /src/hello.c -o /home/hello
/home/hello
```

This check verifies more than the ability to run one generated program. The compiler executing as a TORTOISE user process reproduces the host-built compiler's assembly output. It performs its file I/O through guest system calls, uses the guest library, and writes to a TFS disk through the emulated controller. The system's compiler, assembler, linker, loader, kernel, and CPU agree on one target machine definition.

The `check_nocache` target repeats the program-level tests with the instruction-decoding cache disabled. The ROM monitor and GDB stub have dedicated scripted tests. Snapshot loading, instruction traces, and device-level tests exercise the emulator's observation interfaces separately from the guest-visible behavior.

28. Source size and executable size

The project is organized as source rather than a generated monolithic emulator. In the accompanying tree, the nine `src/*.c` files contain 3239 lines, and the four compiler files in `cc/` contain 3865 lines. The assembler is 1438 lines of C, and `tools/tlink.c` together with `tools/tdisk.c` contain 930 lines. The fifteen C files in `os/kernel/` contain 5118 lines. These counts describe the source present in the archive; assembly, headers, tests, user programs, and the ROM are additional files.

The machine and its guest software are distinct executables. The host compiler builds `tortoise` as a native program, but `tcc0`, the assembler, and the linker also have guest-compiled TEX1 forms stored on the self-hosting disk. A kernel TEX1 file begins with its 32-byte header and linked sections, and each guest user program is another TEX1 file. The file sizes depend on the exact source selection and the target compiler's emitted instructions.

The system's test code is substantial. Eight host-side `tests/test_*.c` programs supply 2695 lines of direct unit and processor tests, while the assembly suite, C tests, operating-system programs, expected transcripts, ROM tests, and self-hosting scripts provide end-to-end coverage. The code volume is distributed across actual components of a complete computer: interpreter, devices, toolchain, kernel, file systems, library, and application environment.

29. Implementation notes

The emulator maintains a precise separation between instruction decoding, architectural execution, and device events. The instruction table supplies format and length; the decoder obtains bytes; the execution switch updates guest registers, flags, memory, and control state. `pg_translate` mediates virtual memory, while `phys_read` and `phys_write` select RAM, ROM, or a memory-mapped device. The decode cache and translation cache improve the host's execution path without changing the instruction-set contract.

The assembler is the canonical encoder for the instruction set. It writes bytes either to a flat output image or to TOF sections with relocations. The compiler targets that assembler: its generator emits readable TORTOISE assembly and relies on assembler and linker passes for symbol references and final section placement. This allows assembly in a C runtime, kernel entry, or disk boot sector to use exactly the same instruction definitions as compiler-generated code.

The boot system follows a series of executable boundaries. The ROM is assembled at `0xFFFF0000`; it loads sector zero; that sector reads a TEX1 kernel from subsequent disk sectors; the kernel entry establishes a C stack and calls `kmain`; and the kernel creates a first user process from a TEX1 executable stored in TFS. No step depends on x86 boot firmware or an external operating-system kernel inside the guest. The entire sequence executes as TORTOISE machine instructions.

The memory system links hardware-level permission checks to operating-system services. Page entries define access rights; CPU faults supply addresses and causes; the kernel maps frames, implements copy-on-write, and returns to the original instruction. Interrupt entry and the common trap frame likewise connect CPU events to the scheduler, syscall interface, and signal machinery. These are concrete interfaces between separately implemented parts, not simulated results returned to an application by the host.

The self-hosting build brings those interfaces together. `tcc0` compiles itself and the assembler and linker into guest executables. Inside the booted OS, these executables use the guest C library and kernel file interface to produce new guest programs. The self-hosting test compares the compiler's generated assembly across the host and in-system executions. It demonstrates that the machine provides the tools and runtime needed to develop software within its own environment.

30. Conclusion

TORTOISE is a complete software-defined computer with a 32-bit processor, backward-compatible legacy instructions, paging and privilege, interrupt and exception delivery, deterministic devices, and a boot ROM. Its emulator executes the defined instruction set, translates virtual addresses, models physical memory and controllers, and can be inspected through tracing, an interactive debugger, and a GDB remote stub.

The toolchain contains an assembler, relocatable TOF object format, symbol-resolving linker, TEX1 executable format, and a C compiler with preprocessing, typed parsing, assembly generation, and a C library. The compiler builds the assembler, linker, and itself for the guest machine. The operating system boots through the guest ROM and disk bootstrap, creates processes, implements copy-on-write and demand paging, handles signals, and serves a filesystem and interactive shell.

The compiled system runs its tools inside its own operating system. It compiles, assembles, links, and executes a new C program through its own kernel and machine instruction interpreter. The in-system compiler emits assembly identical to the corresponding host invocation. TORTOISE therefore provides a complete path from a defined processor and binary format to a self-hosting programming environment.

Availability. The source code of the emulator, assembler, linker, C compiler, libraries, ROM, kernel, user programs, and tests is distributed under the GPL-3.0-only license at:
`https://github.com/gaidardzhiev/tortoise`